

分析主题：EasyDistillation 蒸馏法格点 QCD 软件包（主体结构 / 各部分关系 / 项目思路 / 蒸馏法全流程 A 框架）

opencode analy

2026 年 8 月 17 日

摘要

本报告对被分析仓库 EasyDistillation (格点 QCD 蒸馏法 distillation 软件包, 位于 `refer/git-rep/EasyDistillation/`) 执行完整的只读 analy 分析。分析范围覆盖全库 70 个文件 (58 个 .py、1 个 .cu、4 个 .md、其余配置/数据/前次报告), 全部文件已分配精读/浏览/仅索引三态并登记覆盖表。核心结论: 该库以“拉普拉斯本征向量 → perambulator → elemental → 自动 Wick 收缩 → 关联函数”一条流水线串联 lattice/ 七个 Generator 与收缩引擎, 主体结构上呈“入口/核心/工具/应用/测试/文档”六层; 各部分关系通过 Generator_load/calc 统一接口与惰性 mmap 数据类形成依赖链; 项目思路为多后端 (numpy/cupy/PyQUDA+QUDA) + MPI 任务分发 + 群论自动构造强子算符的一站式封装。主题分析节按 A 代码工作 15 步框架完整重现“蒸馏法全流程”。实测本机环境: 2 个纯逻辑测试套件 (19+8 例) 全部通过, PyQUDA/GPU 数据路径与 LFS 数据未在本机验证。

目录

1 仓库概览	2
1.1 目录结构	2
1.2 规模统计与 git 信息	3
2 主体结构	3
3 各部分关系	4
3.1 依赖主链与数据流	4
3.2 版本演进关系	5
4 项目思路	5
4.1 设计意图	5
4.2 演进脉络	6
4.3 典型工作流	6
5 主题分析：蒸馏法全流程重现（A 代码工作框架）	6
5.1 工作全流程重现（A 代码工作框架）	6
5.1.1 A1 目的与前期准备	6

5.1.2	A2 输入	7
5.1.3	A3 输出应该是什么	7
5.1.4	A4 整体框架	7
5.1.5	A5 关键细节	8
5.1.6	A6 任务如何正确执行	8
5.1.7	A7 实际执行情况	9
5.1.8	A8 实际输出情况	9
5.1.9	A9 结果分析	9
5.1.10	A10 综合分析	9
5.1.11	A11 结尾总结	10
5.1.12	A12 结尾评价	10
5.1.13	A13 补充说明	11
5.1.14	A14 参考源	11
5.1.15	A15 附录与附件	11
6	关键代码/文档片段	11
7	参考源清单	15
8	结论与局限	17

1 仓库概览

1.1 目录结构

```

1 EasyDistillation/
2 |-- README.md TODO.md LICENSE requirements.txt .gitignore _gitattributes
3 |-- doc/README.md          # 数据形状约定 (Gauge/EV/Elemental/Perambulator)
4 |-- docs/                  # 前次 pure 报告 (20260815, 不修改)
5 |-- lattice/               # 核心包 (42 文件)
6 |   |-- __init__.py backend.py constant.py data.py dispatch.py
7 |   |-- hadron.py hadron_irrep.py preset.py quark_contract.py
8 |   |-- quark_diagram.py quark_draw.py
9 |   |-- correlator/        # 4 文件: one_particle/two_particles/disperion_relation
10 |   |-- filedata/         # 6 文件: abstract/binary/ildg/ndarray/sliceloader/timeslice
11 |   |-- generator/        # 9 文件: 7 个 Generator + stout_smear.cu
12 |   |-- insertion/        # 5 文件: gamma/derivative/mom_dict/phase + Operator
13 |   |-- symmetry/         # 7 文件: group_generator/gen_hardcoded_rep/hardcoded_rep...
14 |-- example/              # 8 个使用示例脚本 + README
15 |-- tests/                # 9 个测试脚本 + weak_field.lime (LFS 指针)

```

指标	实测值
文件总数	70 (find 实测)
Python 文件数 / 行数	58 / 20628 (wc -l)
CUDA 文件数	1 (stout_smeas.cu, 245 行)
Markdown 文档数	4
总行数	20981
提交数 (对本目录)	3 (均为主仓库 PyQCD 的导入/收尾提交)
标签数	0

表 1: 仓库实测统计 (数据来源: find/wc/git 实测)

1.2 规模统计与 git 信息

局限与风险

git 信息声明: EasyDistillation/ 内无 .git 目录, 非独立 git 仓库; git log 反映主仓库 PyQCD 的历史。对本目录的 3 个提交为: 4d84e9c (导入本库全套源码与文档)、b683cf0 (.gitattributes 更名 _gitattributes)、e1f7718 (写入前次 pure 报告 PDF)。这些提交 **不构成** 该库自身的设计演进证据, 本报告演进脉络一节仅依据库内 TODO.md、代码注释与源码风格推断 (标注**推断**)。

2 主体结构

蒸馏法软件包的层次分工如下: lattice/ 是唯一核心包, 其内部再按“物理量生成” (generator/), “算符与收缩” (insertion/, quark_*, hadron*), “群论” (symmetry/), “IO” (filedata/, preset.py), “并行” (dispatch.py) 分模块; example/ 与应用/测试层通过 lattice/ 顶层 __init__ 的聚合导出 (lattice/__init__.py:1-32) 接入。

层次	目录	职责
入口层	lattice/__init__.py	聚合导出 7 个 Generator、6 个数据预设类、收缩引擎与常量, 是外部唯一使用入口
核心层	lattice/generator/	7 个 Generator 依次实现蒸馏法四阶段: 拉普拉斯本征向量、perambulator、elemental、密度/广义/噪声 perambulator
核心层	lattice/quark_contract.py	Wick 收缩: 强子味结构 → 传播子乘积 (符号代数 + 邻接矩阵)
核心层	lattice/quark_diagram.py	邻接矩阵 → einsum 收缩图分析 + Diagram 数值计算/化简/批量引擎

层次	目录	职责
核心层	<code>lattice/hadron.py</code> , <code>hadron_irrep.py</code>	强子算符（不可约表示行、味结构）与关联函数生成 <code>gen_correlator</code>
核心层	<code>lattice/correlator/</code>	2pt/双粒子/色散关联函数的直接数值计算
工具层	<code>lattice/insertion/</code>	γ 矩阵（16 组合）、离散导数、动量相位、插入算符 <code>Operator/Insertion</code>
工具层	<code>lattice/symmetry/</code>	O_h^D 与诸小群不可约表示（生成器 + 硬编码表）、Wigner 转动、小群投影
工具层	<code>lattice/filedata/</code> + <code>preset.py</code>	惰性 mmap 文件读取（ILDG/lime、QDP 懒磁盘映射、二进制、numpy）+ 数据类预设
工具层	<code>lattice/backend.py</code> , <code>data.py</code> , <code>dispatch.py</code> , <code>constant.py</code>	多后端抽象、elemental 数据装配、MPI 任务分发、物理常量
应用层	<code>example/</code>	8 个真实使用示例（2pt、双粒子、密度 perambulator、画图等）
测试层	<code>tests/</code>	9 个测试脚本（unittest/脚本式）
文档层	<code>README.md</code> , <code>TODO.md</code> , <code>doc/README.md</code>	定位、路线图、数据形状约定
参考层	<code>docs/</code>	前次 pure 报告（ <code>pure_distillation_20260815.*</code> ）

表 2: 主体结构分层（数据来源: 全库索引实测）

要点

结构小结: 物理量生成 (`generator/`) $\rightarrow$ 算符/收缩 (`insertion/` + `quark_*/hadron*`) $\rightarrow$ 关联函数 (`correlator/`) 构成唯一纵向主线; `filedata/preset/backend/ dispatch` 是贯穿全线的横向工具带。

3 各部分关系

3.1 依赖主链与数据流

蒸馏法软件包的各部分以“数据对象”为纽带串联: 数据类 (`preset.py`) 提供统一 `load(key)` 读取接口, `Generator` 类提供统一 `load/calc` 接口, 收缩引擎以邻接矩阵为中间表示。

要点

依赖主链: `preset`(数据类) $\rightarrow$ `generator` (`load_Eigenvector` $\rightarrow$ `Perambulator` $\rightarrow$ `Elemental`) $\rightarrow$ `insertion` (`Operator/Insertion`) $\rightarrow$ `hadron` (`gen_correlator`) $\rightarrow$ `quark_contract` (Wick) $\rightarrow$ `quark_diagram` (`QuarkDiagram` $\rightarrow$ `compute_diagrams`) $\rightarrow$ `correlator` (2pt)

- **数据类** → **物理量**: preset.py 中 GaugeFieldTimeSlice:44-53、EigenvectorTimeSlice:55-63、PerambulatorBinary:92-100、ElementalNpy:173-181 分别是四类数据的“形状 + 文件前缀 + 读取”封装; doc/README.md:3-65 给出四类数据形状约定 (gauge [Lt, Ls³, Ns, Nc, Nc]、eigenvector [Lt, Ne, Ls³, Nc]、elemental [nd, nmom, Lt, Ne, Ne]、perambulator [Lt, Lt, Ns, Ns, Ne, Ne])。
- **Generator** → **数据**: generator/__init__.py:1-7 聚合导出 7 个 Generator; 每个 Generator 的 load(key) 调数据类、calc(t) 做数值, 例如 eigenvector.py:49-53 (load 转置拷贝)、elemental.py:102-105。
- **收缩引擎** → **算符**: hadron.py:17-19 从 symmetry/、quark_contract.py、quark_diagram.py 导入; hadron.py:95-151 的 gen_correlator 把味波函数交给 quark_contract (:141), 把结果顶替算符索引 (:146)。
- **并行带**: dispatch.py:62-103 的 Dispatch 以文件为任务队列 (rank 0 逐行消费 + MPI broadcast), example 中 gen_two_particle_corr_mom.py:104-105 直接 for cfg in dispatcher 消费。

3.2 版本演进关系

该库无独立 git 演进记录; 从 TODO.md:3-13 与源码注释可见内部功能进度: 已勾选项为 stout 平滑 NDAarray/CUDA 实现 (:3-4)、PyQuda perambulator (:6)、群论乘积 (:10)、2 粒子相关函数含 P/D 波 (:12)、3pt 表示 (:13); 未勾选项为 API 一致性 (:5)、顺序源 (:7)、log 系统 (:8)、helicity 算符 (:11)。推断: 该库处于“功能先行、工程化滞后”的科研代码形态。

4 项目思路

4.1 设计意图

README.md:3-7 明确目标: 把蒸馏法的“拉普拉斯本征向量、perambulator、elemental、自动收缩”一体化集成, 以 Generator 类追求灵活高效。设计选择可从代码验证:

- **多后端**: backend.py:14-33 支持 numpy/cupy (torch 留注释), get_backend() 惰性默认 numpy (:7-11); PyQUDA 存在性探测 check_QUDA:36-58。
- **惰性 IO**: filedata 一律 mmap 按需切片读取并统计吞吐 (binary.py:36-62、ildg.py:49-73), 避免把大数组全部载入显存。
- **统一 load/calc**: 所有 Generator 同构, 配合“计算与 IO 分离”, 便于 MPI 多组态批处理。
- **群论自动化**: 强子不可约表示 ($A_1/A_2/E/T_1/T_2/G/H$) 与各动量小群不可约表示预生成/硬编码 (group_generator.py:5-198、hardcoded_rep.py:10188), 使“物理上该有的算符组合”自动枚举而非手写。

4.2 演进脉络

- **外部演进**：本目录于 2026-08-16 随主仓库 4d84e9c 整体导入（无历史提交），故无版本化演进证据。**推断**：软件本身的演进只能从代码内部痕迹读取。
- **内部痕迹**：elemental.py:301-308 中朴“素逐项求和被”“2 的幂划分左/右导数”写法取代（注释掉的旧分支）；quark_diagram.py:330-553 的 Diagram.simplify 说明在“冗余顶点删除 → 顶点/传播子排序 → 连通分量拆分”上迭代过；insertion/__init__.py:255-267 的 little_group_projection 与小群不可约化表明算符构建从手写向群论自动化迁移（TODO.md:10 对应）。

4.3 典型 workflow

以 example/gen_twopt.py:1-68 为代表的单粒子 2pt workflow:

1. set_backend("cupy") 选择后端 (:3);
2. 用 Insertion(GammaName.PI, DerivativeName.IDEN, ProjectionName.A1, momDict) 构建 π 插入算符并转成 Operator (:9-11);
3. 实例化 ElementalNpy 与 PerambulatorNpy 并 load(cfg) (:21-36);
4. 调 twopoint (correlator/one_particle.py:12-77) 得 [Nop, Lt] 2pt, 再 arccosh 提取有效质量 (:45-47)。

双粒子 workflow (gen_two_particle_corr_mom.py:100-175) 则在收缩阶段改用 QuarkDiagram 邻接矩阵 + compute_diagrams_multitime 直接组装关联函数。

项目思路

项目思路核心总结：这个仓库把“蒸馏法”从“手写脚本”抽象为 Generator 流水线 + 数据类统一 IO + 图论化 Wick 收缩 + 群论算符自动构建”四件套，目标是让物理工作者只写“算符定义 + 数据路径”即可拿到关联函数。

5 主题分析：蒸馏法全流程重现（A 代码工作框架）

主题为“蒸馏法全流程”——从规范场出发经本征向量、perambulator、elemental 到关联函数的一次完整计算。判定为**代码/软件工作**，采用 A 框架；物理输入（公理/模型）并入相应步骤。15 步编号小节齐全，缺证据步骤如实标注。

5.1 工作全流程重现（A 代码工作框架）

5.1.1 A1 目的与前期准备

目的：蒸馏法 (distillation) 用拉普拉斯算符最低若干本征向量展开夸克场，把“3+1 维格点自由度”压缩为“ N_e 维蒸馏空间”，从而以传播子矩阵 (perambulator) 与张量 (elemental) 做收缩，显著降低

Wick 收缩成本。该库把这一流程打包为一站式 Python 包 (README.md:3-7)。**前置条件**: 纯 CPU 路径仅需 requirements.txt:1-4 (numpy/scipy/sympy/opt-einsum); GPU/反演路径需 PyQUDA + QUDA (README.md:14) 且后端为 cupy (perambulator.py:77); 多组态并行需 mpi4py (dispatch.py:6)。**环境实测**: 本机 numpy 2.1.2/scipy 1.14.1/sympy 1.14.0/opt-einsum 可导入; cupy 有 1 张可用 GPU; pyquda 导入失败 (缺 libquda.so)。**未验证**: GPU 反演路径本机不可运行。

5.1.2 A2 输入

输入	说明与参考源
规范场	Chroma/QDP 生成的 .lime 文件 (GaugeFieldIldg, preset.py:151-159) 或 QDP 懒磁盘映射 .mod (GaugeFieldTimeSlice:44-53) ; 数据形状 [Lt,Lz,Ly,Lx,Nd,Nc,Nc]
拉普拉斯本征向量	由 laplace_eigs 生成的 .mod (EigenvectorTimeSlice:55-63) , 形状 [Lt,Ne,Lz*Ly*Lx,Nc] (doc/README.md:25-27)
夸克参数	mass/tol/maxiter/xi_0/nu/clover_coeff_t/r/t_boundary/multigrid/MRHS (perambulator.py:51-68, 实例见 test_perambulator.py:29-44)
蒸馏空间参数	本征向量数 Ne、求解容差 tol (test_eigenvector.py:16-20)
算符	γ 组合 (gamma.py:92-100 的 16 个二进制编号) 、导数组合 (derivative.py:23-33 的 3 进制编号)、动量列表 (mom_dict.py:1-159)
物理参数	光/粲质量、t_boundary (周期/反周期)

表 3: A2 输入清单 (数据来源: preset.py/doc README 实测)

5.1.3 A3 输出应该是什么

按阶段输出 (形状见 doc/README.md:3-65 与 preset.py):

- **平滑规范场**: stout 平滑 n 步 (eigenvector.py:215-232) 后的 $U_\mu(x)$;
- **本征向量**: 每时间片 [Ne, Lz, Ly, Lx, Nc] (eigenvector.py:257), 并做重整相位 (:254-256);
- **perambulator**: [Lt, Lt, Ns, Ns, Ne, Ne] 蒸馏空间传播子 (perambulator.py:102) , $\tau(t_f, t_i)_{jk}^{SS'} = \langle v_j | D^{-1} | v_k \rangle$;
- **elemental**: [num_deriv, num_mom, Lt, Ne, Ne] (elemental.py:56), $V^\dagger(\nabla^n) e^{ip \cdot x} V$;
- **关联函数**: 2pt [Nop, Lt] (one_particle.py:51), 最终乘以 -1 为约定符号 (:74-77)。

5.1.4 A4 整体框架

主流程四阶段与模块映射:

- **阶段 1 (本征向量)**: generator/eigenvector.py, 内部再分 scipy/cupyx 求解、QUDA 求解、stout 平滑三路径 (:356-370);

- 阶段 2 (perambulator): `generator/perambulator.py`, PyQUDA 反演 $Dv = \text{源}$ (:146-209); 广义/密度变体见 `generalized_perambulator.py`, `density_perambulator.py`;
- 阶段 3 (elemental): `generator/elemental.py` 离散协变导数 + 动量相位投影 (:290-338);
- 阶段 4 (收缩): `quark_contract.py` (Wick) $\rightarrow$ `quark_diagram.py` (einsum 计算) $\rightarrow$ `correlator/` (2pt/双粒子)。

入口统一在 `lattice/__init__.py`:1-32 聚合。

5.1.5 A5 关键细节

- **拉普拉斯算符**: `eigenvector.py`:11-26 的 `_Laplacian` 用 `opt_einsum.contract` 实现三维求和 $\sum_{\mu}(U_{\mu}(x)\psi(x+\hat{\mu}) + U_{\mu}^{\dagger}(x-\hat{\mu})\psi(x-\hat{\mu}))$, 对角项取 6 (SA 求最小本征)。
- **stout 平滑**: `eigenvector.py`:123-190 实现 Morningstar-Peardon 指数化 e^{iQ} (含三阶展开 f0/f1/f2、奇偶 parity 修正); CUDA 核 `stout_smear.cu`:160-245 逐链接计算 staple 后矩阵指数。
- **递归 Wick 收缩**: `quark_contract.py`:216-245 `_quark_contract` 以反夸克为锚 (:221-222) 递归配对同位素夸克, 用 $(-1)^{i-j}$ 计入交换符号, u/d 简并时统一用味 "q" (:233-236)。
- **邻接矩阵 $\rightarrow$ einsum**: `quark_diagram.py`:23-79 `QuarkDiagram.analyse` 用 BFS 分解连通分量, 为每个传播子分配张量下标 (`_SUB_M` 自旋/ `_SUB_A` 色空间), 自动生成 `contract` 表达式 (:79)。
- **γ_5 -厄米性**: 反向传播子用 $\gamma_5\tau^{\dagger}\gamma_5$ (`one_particle.py`:57), 源/汇算符用 γ_4 共轭 (:63)。

5.1.6 A6 任务如何正确执行

1. 数据准备: 生成/获取 `.lime` 规范场与 `.mod` 本征向量 (外部工具), 数据形状必须与 `doc/README.md` 一致;
2. 设置后端 `set_backend("numpy"|"cupy")` (`example/gen_twopt.py`:3);
3. `EigenvectorGenerator`: `load` $\rightarrow$ `stout_smear` $\rightarrow$ 逐 t 片 `calc(t)` (`test_eigenvector.py`:20, 44-50);
4. `ElementalGenerator`: `load` $\rightarrow$ 逐 t 片 `calc(t)` (`test_elemental.py`:26, 41-45);
5. `PerambulatorGenerator` (需 PyQUDA): `load` $\rightarrow$ 逐 t 片 `calc(t)` (`test_perambulator.py`: 29-44, 62-64);
6. 定义插入算符 $\rightarrow$ 调用 `twopoint/twopoint_matrix` (`example/gen_twopt.py`:40-51) 或双粒子图收缩 (`example/gen_two_particle_corr_mom.py`:162-168)。

5.1.7 A7 实际执行情况

本机实测 (记录于会话日志 `.analy.*.log`):

- `python3 tests/test_quark_contract.py` → Ran 19 tests in 0.012s, OK;
- `python3 tests/test_simplify.py` → Ran 8 tests in 0.039s, OK;
- `python3 -c "import lattice"` → OK (mpi4py 4.1.1 可用);
- `import cupy` → OK, 1 张 GPU 设备;
- `import pyquda` → ImportError: libquda.so 缺失;
- `python3 tests/test_load.py` → FileNotFoundError (`weak_field.npy` 为 LFS 指针, 数据未拉取)。

结论: 纯逻辑链路 (收缩/化简) 可完整执行; 数据依赖链路 (IO/本征求解/反演) 本机缺 LFS 数据与 QUDA 库, 未执行。

5.1.8 A8 实际输出情况

`test_quark_contract` 与 `test_simplify` 各用例输出均断言通过 (`test_quark_contract.py`: 153-222、`test_simplify.py`: 72-367)。手工冒烟: meson-meson ($\bar{u}d \times \bar{d}u$) 收缩输出 `-...q` 形式并得邻接矩阵 `[[1,0],[0,1]]`, 与 `test_quark_contract.py`:184 的断言一致。数据相关测试的输出 (本征值残差、perambulator 范数等) 本机未产生。

5.1.9 A9 结果分析

对已生成的实测结果逐项解读:

- 19+8 例逻辑测试通过, 说明 Wick 收缩的符号约定 ($(-1)^{i-j}$)、u/d 简并替换、邻接矩阵编码、图化简 (连通分量拆分 `quark_diagram.py`: 330-553) 在符号层自治;
- `test_simplify.py`: 74-206 验证了“顶点排序、冗余顶点删除、连通分量拆分”三个化简不变量——不同顶点排列/时标顺序化简后得到同一正则形式, 这是批量计算去重的基础 (`calc_diagram` 依据 `Diagram.__eq__` 合并相同图, `quark_diagram.py`: 317-328);
- 未生成的数值类输出 (有效质量平台、色散) 无法在本机核对, 标注**未验证**; `gen_twopt.py`: 47, 52-61 中有效质量公式为 $\cosh^{-1}((C(t-1) + C(t+1))/2C(t))$, 是标准格点做法。

5.1.10 A10 综合分析

与仓库整体联系: 蒸馏法全流程横跨**核心层全部四个子模块**, 是 `lattice/` 六层结构的“总线”。代码-对象映射:

代码符号	物理对象	物理/业务含义
GaugeField	规范场 $U_\mu(x)$	格点 SU(3) 联络 (preset.py:11-14)
_Laplacian	三维协变拉普拉斯 ∇^2	本征方程 $\nabla^2\psi = \lambda\psi$ 本征空间展开基底 (eigenvector.py:11-26)
EigenvectorGenerator	拉普拉斯本征向量 $v^{(k)}(x)$	蒸馏基底 (本征值升序, 重整相位, eigenvector.py:234-257)
Perambulator	蒸馏传播子 $\tau = D^{-1} _V$	夸克传播子在蒸馏空间的矩阵元 (perambulator.py:102,198)
Elemental	张量 $\Phi = V^\dagger \Gamma \nabla^n e^{ipx} V$	算符在蒸馏空间矩阵元 (elemental.py:324-329)
gamma(n)	γ 矩阵 Γ	16 个基的组合编号 (gamma.py:92-100)
derivative(n)	离散导数 ∇^n	3 进制方向编码 (derivative.py:23-33)
MomentumPhase	平面波 $e^{ip \cdot x}$	动量投影相位 (phase.py:41-46)
Operator/Insertion	插值算符 $\mathcal{O}_\Gamma(\nabla, p)$	强子量子数定义 (insertion/__init__.py:115-168)
QuarkDiagram	Wick 收缩图	传播子连接拓扑的邻接矩阵 (quark_diagram.py:15-79)
Diagram	收缩项	一张收缩图对应的数值项 (quark_diagram.py:281-328)
compute_diagrams	关联函数	einsum 求值各图 (quark_diagram.py:261-273)

表 4: 蒸馏法全流程代码-对象映射 (数据来源: 源码逐符号核验)

5.1.11 A11 结尾总结

结论

蒸馏法全流程结论: 该流程在库内被封装为**四阶段 Generator 流水线**, 每阶段 load/calc 同构; 从规范场到 2pt 关联函数全程无需手写传播子, “Wick 收缩自动化为邻接矩阵、数值计算自动化为 einsum” 是最大抽象点; 纯逻辑部分本机实测通过, GPU/反演部分依赖 PyQUDA+QUDA 与 LFS 数据 (未验证)。

5.1.12 A12 结尾评价

优点: (1) 自动 Wick 收缩把易错的置换符号交给符号代数; (2) 多后端与惰性 mmap IO 兼顾 CPU/GPU 与大数据; (3) 群论自动枚举算符组合减少手写重复; (4) MPI Dispatch 支持多组态批处理。**可改进:** (1) TODO.md:5 自述 API 尚未完全一致; (2) examples/tests 含大量硬编码绝对路径 (如

gen_twopt.py:22); (3) 数据依赖 LFS 使仓库克隆后无法直接复现 (_gitattributes:1-9); (4) 版本兼容硬约束在 backend.py:46-47 (PyQUDA 必须 $\geq 0.9.0$)。

5.1.13 A13 补充说明

假设/局限/未覆盖: GPU 求解路径 (cupyx、QUDA)、MPI 多进程路径、stout CUDA 核的实际数值结果均未在本机运行 (缺 QUDA/LFS 数据), 标注**未验证**; hardcoded_rep.py 11613 行仅抽样定位结构 (浏览态); 统计数字均以本机实测为准。无相关依据项: 库内无基准测试/性能数据文件 (未找到相关依据)。

5.1.14 A14 参考源

本步证据汇总见第 7 节参考源清单; 关键: README.md:3-7、TODO.md:3-13、doc/README.md:3-65、lattice/___init___py:1-32、backend.py:14-33、preset.py:44-181、filedata/binary.py:36-62、eigenvector.py:11-26/234-257/356-370、elemental.py:48-56/290-338、perambulator.py:102/146-209、quark_contract.py:110-245、quark_diagram.py:15-79/233-273、hadron.py:95-151、insertion/___init___py:115-350、one_particle.py:12-77、example/gen_twopt.py:1-68、tests 实测结果。

5.1.15 A15 附录与附件

实测命令与输出记录于会话日志 .analy.2026-08-17-06-18-21.log; 本机环境: numpy 2.1.2、scipy 1.14.1、sympy 1.14.0、mpi4py 4.1.1、cupy(1 GPU)、pyquda 导入失败 (缺 libquda.so)。

6 关键代码/文档片段

- 拉普拉斯算符 (eigenvector.py:11-26):

```

1 def _Laplacian(F, U, U_dag, latt_size):
2     backend = get_backend()
3     Lx, Ly, Lz, Lt = latt_size
4     F = F.reshape(Lz, Ly, Lx, Nc, -1)
5     return (
6         # - for SA with evals , + for LA with (12 - evals)
7         6 * F
8         - (
9             contract("zyxab,zyxbc->zyxac", U[0], backend.roll(F, -1, 2))
10            + contract("zyxab,zyxbc->zyxac", U[1], backend.roll(F, -1, 1))
11            + contract("zyxab,zyxbc->zyxac", U[2], backend.roll(F, -1, 0))
12            + backend.roll(contract("zyxab,zyxbc->zyxac", U_dag[0], F), 1, 2)
13            + backend.roll(contract("zyxab,zyxbc->zyxac", U_dag[1], F), 1, 1)
14            + backend.roll(contract("zyxab,zyxbc->zyxac", U_dag[2], F), 1, 0)
15        )
16    ).reshape(Lz * Ly * Lx * Nc, -1)

```

- 求解路径分发 (eigenvector.py:356-370):

```

1 def calc(self, t: int, apply_renorm_phase: bool = True, polynomial_degree: int = 0, lambda_cut: float = 0.0):
2     backend = get_backend()
3     # Don't use QUDA's eigensolver because of some performance regression.
4     if backend.__name__ == "cupy" and check_QUDA():
5         if polynomial_degree > 0 and lambda_cut > 0:
6             print("Using quda Laplacian and quda solver with Chebyshev polynomial acceleration.")
7             return self.laplacian_quda(t, apply_renorm_phase, polynomial_degree, lambda_cut)
8         else:
9             print(f"Using quda Laplacian and {backend.__name__} solver.")
10            return self.laplacian_quda_scipy(t, apply_renorm_phase)
11     elif backend.__name__ == "cupy" or backend.__name__ == "numpy":
12         print(f"Using {backend.__name__} Laplacian and {backend.__name__} solver.")
13         return self.laplacian_cupy_numpy(t, apply_renorm_phase)
14     else:
15         raise NotImplementedError(f"Unsupport backend = {backend.__name__}.")

```

- 递归 Wick 收缩 (quark_contract.py:216-245):

```

1 def _quark_contract(symbol_list, result_list, result, degenerate):
2     if symbol_list == []:
3         result_list.append(Mul(*result))
4         return
5     for i, src in enumerate(symbol_list):
6         if src.anti:
7             break
8     for j, snk in enumerate(symbol_list):
9         if not snk.anti and snk.flavor == src.flavor:
10             if i > j:
11                 symbol_list.pop(i)
12                 symbol_list.pop(j)
13                 factor = S(-1) ** (i - j - 1)
14             else:
15                 symbol_list.pop(j)
16                 symbol_list.pop(i)
17                 factor = S(-1) ** (j - i)
18             if degenerate and (snk.flavor == "u" or snk.flavor == "d"):
19                 prop = Propagator("q", src.tag, snk.tag)
20             else:
21                 prop = Propagator(src.flavor, src.tag, snk.tag)
22             result.append(factor * prop)
23             _quark_contract(symbol_list, result_list, result, degenerate)
24             result.pop()
25             if i > j:
26                 symbol_list.insert(j, snk)
27                 symbol_list.insert(i, src)
28             else:
29                 symbol_list.insert(i, src)
30                 symbol_list.insert(j, snk)

```

- 邻接矩阵 → einsum (quark_diagram.py:23-60):

```

1 def analyse(self) -> None:

```

```

2     from copy import deepcopy
3
4     adjacency_matrix = deepcopy(self.adjacency_matrix)
5     num_vertex = len(adjacency_matrix)
6     visited = [False] * num_vertex
7     for idx in range(num_vertex):
8         if visited[idx]:
9             continue
10        propagators = []
11        visited[idx] = True
12        queue = [idx]
13        while queue != []:
14            i = queue.pop(0)
15            for j in range(num_vertex):
16                path = adjacency_matrix[i][j]
17                if path != 0:
18                    adjacency_matrix[i][j] = 0
19                    if not visited[j]:
20                        visited[j] = True
21                        queue.append(j)
22                    if isinstance(path, int):
23                        propagators.append([path, i, j])
24                    elif isinstance(path, list):
25                        for _path in path:
26                            propagators.append([_path, i, j])
27                    else:
28                        raise ValueError(f"Invalid value {path} in the adjacency matrix")
29        if propagators == []:
30            continue
31        vertex_operands = []
32        vertex_subscripts = []
33        propagator_operands = []
34        propagator_subscripts = []
35        node = 0
36        for propagator in propagators:
37            propagator_operands.append(propagator)
38            propagator_subscripts.append(_SUB_M[node + 1] + _SUB_A[node + 1] + _SUB_M[node] + _SUB_A[node])

```

- elemental 计算 (elemental.py:297-336):

```

1     for e in range(V.shape[0]):
2         V[e] = eigenvector[t, e]
3     for derivative_idx, derivative in enumerate(self.derivative_list):
4         VPV[derivative_idx] = 0
5         # for num_nabla_right in range(len(derivative) + 1):
6         #     coeff = (-1)**num_nabla_right * comb(len(derivative), num_nabla_right)
7         #     right = self._nD(V, U[:, t], derivative[:num_nabla_right])
8         #     left = self._nD(V, U[:, t], derivative[num_nabla_right:][::-1])
9         #     for momentum_idx, momentum in enumerate(self.momentum_list):
10            #         VPV[derivative_idx, momentum_idx] += contract(
11            #             "zyx,ezyxc,fzyxc->ef", coeff * momentum_phase.get(momentum), left.conj(), right
12            #         )
13        for pick_nabla in range(2 ** len(derivative)):
14            pick_right = []

```

```

15     pick_left = []
16     pick = pick_nabla
17     for direction in derivative:
18         if pick & 1:
19             pick_right.append(direction)
20         else:
21             pick_left.append(direction)
22         pick >>= 1
23     coeff = (-1) ** len(pick_right)
24     right = self._nD(V, U[:, t], pick_right)
25     left = self._nD(V, U[:, t], pick_left[::-1])
26     for momentum_idx, momentum in enumerate(self.momentum_list):
27         if self.stochastic_coeff is None:
28             VPV[derivative_idx, momentum_idx] += contract(
29                 "zyx,ezyxc,fzyxc->ef",
30                 coeff * momentum_phase.get(momentum),
31                 left.conj(),
32                 right,
33             )
34         else:
35             VPV[derivative_idx, momentum_idx] += contract(
36                 "zyx,ezyxc,fzyxc,ef->ef",
37                 coeff * momentum_phase.get(momentum),
38                 left.conj(),
39                 right,
40                 self.stochastic_coeff,

```

- 2pt 收缩 (one_particle.py:53-77):

```

1     for it, t_src in enumerate(timeslices):
2         tau = perambulator[t_src, :, :, :, :usedNe, :usedNe]
3         # tau = backend.roll(perambulator[t, :, :, :, :usedNe, :usedNe], -t, 0)
4         if perambulator_bw is None:
5             tau_bw = contract("ii,tjiba,jj->tijab", gamma(15), tau.conj(), gamma(15))
6         else:
7             tmp = perambulator_bw[t_src, :, :, :, :usedNe, :usedNe]
8             tau_bw = contract("ii,tjiba,jj->tijab", gamma(15), tmp.conj(), gamma(15))
9         for idx in range(Nop):
10             phi = phis[idx]
11             gamma_src = contract("ij,xkj,kl->xil", gamma(8), phi[0].conj(), gamma(8))
12             ret[idx, it] = contract(
13                 "tijab,xjk,xtbc,tklcd,yli,yad->t",
14                 tau_bw,
15                 phi[0],
16                 backend.roll(phi[1], -t_src, 1),
17                 tau,
18                 gamma_src,
19                 phi[1][:, t_src].conj(),
20             )
21             # print(f"t{t}: {perambulator.size_in_byte/perambulator.time_in_sec/1024**2:.5f} MB/s")
22     if is_sum_over_source_t:
23         return -ret.mean(1)
24     else:
25         return -ret

```

- perambulator 反演 (perambulator.py:146-175):

```
1 def calc(self, t: int):
2     import cupy as cp
3
4     backend = get_backend()
5     from pyquda_utils.core import LatticeFermion, MultiLatticeFermion
6
7     if self.gauge_field_new:
8         self.dirac.loadGauge(self.gauge_field_smear) # loadGauge after
9         self.gauge_field_new = False
10
11     latt_info = self.latt_info
12     Lx, Ly, Lz, Lt = latt_info.size
13     Vol = Lx * Ly * Lz * Lt
14     Ne = self.usedNe
15     eigenvector_dagger = self._eigenvector_data_dagger
16     dirac = self.dirac
17     gx, gy, gz, gt = self.latt_info.grid_coord
18
19     SV = self._SV
20     VSV = self._VSV
21
22     from time import perf_counter
23
24     for eigen in range(Ne):
25         cp.cuda.runtime.deviceSynchronize()
26         s = perf_counter()
27         if self.MRHS:
28             print("Warning: use MRHS.")
29             V_MRHS = MultiLatticeFermion(latt_info, Ns)
30             for spin in range(Ns):
```

7 参考源清单

参考源	类型	内容/作用
README.md:3-7	仓库	包定位：蒸馏法四阶段一站式集成
README.md:14	仓库	本征求解与 stout 平滑依赖 PyQuda+QUDA
TODO.md:3-13	仓库	演进路线：已完成/未成功能
doc/README.md:3-65	仓库	四类数据形状约定
requirements.txt:1-4	仓库	依赖：numpy/scipy/sympy/opt-einsum
lattice/__init__.py:1-32	仓库	顶层聚合导出（唯一入口）
lattice/backend.py:14-33	仓库	set_backend 多后端切换
lattice/backend.py:36-58	仓库	check_QUDA 探测 PyQUDA

参考源	类型	内容/作用
<code>lattice/preset.py:44-181</code>	仓库	数据类预设 (Gauge/EV/Peram/Elemental)
<code>lattice/filedata/binary.py:36-62</code>	仓库	mmap 惰性切片读取 + 吞吐统计
<code>lattice/filedata/ildg.py:49-73</code>	仓库	ILDG/lime 格式解析读取
<code>lattice/filedata/timeslice.py:36-99</code>	仓库	QDP 懒磁盘映射读取
<code>lattice/dispatch.py:62-103</code>	仓库	MPI 任务分发 (文件队列 + broadcast)
<code>lattice/generator/eigenvector.py:11-26</code>	仓库	三维协变拉普拉斯
<code>lattice/generator/eigenvector.py:234-257</code>	仓库	scipy/cupyx 本征求解路径
<code>lattice/generator/eigenvector.py:259-312</code>	仓库	QUDA 本征求解路径
<code>lattice/generator/eigenvector.py:356-370</code>	仓库	calc 求解路径分发
<code>lattice/generator/stout_smear.cu:160-245</code>	仓库	stout 平滑 CUDA 核
<code>lattice/generator/elemental.py:48-56</code>	仓库	导数/动量维度构造
<code>lattice/generator/elemental.py:290-338</code>	仓库	elemental VPV 计算
<code>lattice/generator/perambulator.py:51-102</code>	仓库	PerambulatorGenerator 构造
<code>lattice/generator/perambulator.py:146-209</code>	仓库	PyQUDA 反演 + 收缩
<code>lattice/generator/noisevector.py:35-110</code>	仓库	随机蒸馏噪声向量
<code>lattice/insertion/gamma.py:92-100</code>	仓库	γ 矩阵 16 组合构造
<code>lattice/insertion/derivative.py:23-33</code>	仓库	离散导数 3 进制编码
<code>lattice/insertion/phase.py:41-53</code>	仓库	动量平面波相位 (含 cb2)
<code>lattice/insertion/__init__.py:115-168</code>	仓库	Operator 算符组装
<code>lattice/insertion/__init__.py:219-350</code>	仓库	Insertion 算符 + 小群投影
<code>lattice/symmetry/group_generator.py:5-198</code>	仓库	O_h^D /小群不可约表示生成器
<code>lattice/symmetry/gen_hardcoded_rep.py:162-223</code>	仓库	小群不可约表示/硬编码索引
<code>lattice/symmetry/hardcoded_rep.py:10188</code>	仓库	OD 不可约表示硬编码表
<code>lattice/quark_contract.py:110-213</code>	仓库	主收缩: 味结构 $\rightarrow$ 传播子 + 邻接矩阵
<code>lattice/quark_contract.py:216-245</code>	仓库	递归配对收缩 (符号/简并)
<code>lattice/quark_diagram.py:15-79</code>	仓库	QuarkDiagram 邻接矩阵 $\rightarrow$ einsum
<code>lattice/quark_diagram.py:233-273</code>	仓库	compute_diagrams(数值求值
<code>lattice/quark_diagram.py:330-553</code>	仓库	Diagram.simplify 图化简
<code>lattice/quark_diagram.py:785-960</code>	仓库	calc_diagram 批量去重计算
<code>lattice/hadron.py:95-151</code>	仓库	gen_correlator 关联函数组装
<code>lattice/hadron_irrep.py:20-158</code>	仓库	强子不可约表示类
<code>lattice/correlator/one_particle.py:12-77</code>	仓库	twopoint 2pt 计算
<code>lattice/correlator/one_particle.py:223-273</code>	仓库	twopoint_isoscalar 味单态
<code>lattice/correlator/disperion_relation.py:10-32</code>	仓库	固定 mom^2 2pt (色散)
<code>example/gen_twopt.py:1-68</code>	仓库	单粒子 2pt 典型工作流

参考源	类型	内容/作用
example/gen_two_particle_corr_mom.py:100-175	仓库	双粒子动量扫描 workflow
tests/test_quark_contract.py:145-222	仓库	收缩 19 例测试 (本机通过)
tests/test_simplify.py:72-367	仓库	化简 8 例测试 (本机通过)
tests/weak_field.lime	仓库	LFS 指针 (131B), 数据未拉取
_gitattributes:1-9	仓库	LFS 数据文件登记
LICENSE:1-21	仓库	MIT 许可 (2023 developers)

8 结论与局限

结论

结构：六层（入口/核心/工具/应用/测试/文档）分层清晰，核心层为“generator 四阶段流水线 + quark_* 收缩引擎 + correlator 数值”；**关系：**以数据类 load 与 Generator load/calc 统一接口织网，数据流方向唯一（规范场 →EV→peram→elemental→ 关联函数）；**思路：**多后端 + 惰性 mmap IO + 群论自动算符 + MPI 分发的科研型一站式封装，目标是“定义算符 + 给数据路径即得关联函数”。**主题：**蒸馏法全流程 15 步完整重现，纯逻辑链路本机实测通过（27 例），GPU/QUDA 与 LFS 数据路径未验证。

局限与风险

局限与风险：(1) GPU 求解路径、MPI 路径、stout CUDA 核数值结果未在本机运行（缺 libquda.so 与 LFS 数据），**未验证**；(2) 该库无独立 git 历史，演进脉络仅为代码内部痕迹推断；(3) hardcoded_rep.py 为浏览态（抽样定位）；(4) 示例与测试含硬编码路径，克隆后需按实际数据路径修改才能复现；(5) pyquda 版本强约束（≥0.9.0）在 backend.py:46-47。